

Introduction

The question of optimal trading strategies has persisted for as long as finance itself. As such we have seen the application of new machine learning methods applied to this domain. Reinforcement Learning (RL) in the context of finance is not new, and there is still recent works in this field, including those involving Multi-Agent RL (MARL) [1][2]. Often times they may take multiple agents together to tackle a problem, but one can also explore game theoretical relations between traders [3]. To this end, one can consider a set of traders competing against each other with the goal of achieving the best returns they can by trading shares of a single asset. However, one thing to note when dealing with trading is its effects on the prices of the assets, which we can describe as market impact [4]. With this in mind, can agents learn how to trade a assets when their learning and trading affects its price? Are some agents better than others in this aim? Such questions are important, as the ability to find optimal strategies will be effect by such impact. Hence, these are the questions we intend to address in this project.

Objectives and Methodology

Our goal is to examine how well agents can learn how to trade an asset in the presence of market effects. To achieve this end, we consider various agents $1, \dots, n$ with different trading and learning strategies. We consider just one asset $X = (X_t)_{t=0}^T$ as a discrete time stochastic process with which the agents will trade, whose price fluctuations change based on how much it gets traded. We will train all of these models simultaneously, and will evaluate their ability to find optimal strategies with market effects present.

For simplicity, the asset will have 100 shares and trading of non-integer quantities of shares will not be possible. Let δ_t^i be the number of shares owned by agent i at time t , and let $\Delta_t = \sum_{i=1}^n \delta_t^i$ be the number of total shares that are in circulation at time t . At times $t = 0, \dots, T$ the asset price is defined as

$$X_t = \left[1 - \frac{1}{100} U(-\Delta_t, \Delta_t) \right] x_t$$

where $U(-\Delta_t, \Delta_t)$ is a random variable which uniformly picks values from $\{-\Delta_t, -\Delta_t + 1, \dots, -\Delta_t - 1, \Delta_t\}$. Essentially, we have a fixed discrete time series $x = (x_t)_{t=0}^T$ whose random price fluctuations are brought about by random scaling. It is clear that the price changes can become more severe as the number of shares in circulation increases. In reality, while there is information open to all agents, there is personal information for which only the trader themselves would know, and thus the state spaces will be agent dependent. For a given agent i the state space is $\mathcal{S}^i = \{(t, \Delta_t, \delta_t^i, u) \mid u \in \{-1, 0, 1\}\}$, with u being -1, 0 or 1 if the stock price has decreased, remained the same or increased respectively. The initial states are $S_0^i = (0, 0, 0, 0)$ and the terminal states are $S_T^i \in \{(T, \Delta_T, \delta_T^i, u) \mid u \in \{-1, 0, 1\}\}$. At a given time step, the agent can request to buy or sell up to 100 shares, and thus the action space is $\mathcal{A} = \{-100, -99, \dots, -1, 0, 1, \dots, 99, 100\}$. Of course, the request may not be fully met. If you try to buy more than $\lfloor \frac{1}{n} \rfloor$ percent of available shares, you will only get $\lfloor \frac{1}{n} \rfloor$ percent of available shares. If you try to sell more shares than you have, you will only sell as many shares as you have. The reward function $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is defined as $R(S, A) = X_t$.

The agents will have two main methods for learning: Q-Learning and Deep Q-Network. We have $\mathcal{H} = \{(\eta = (\alpha, \epsilon_0, r), \theta = (L, N, a))\}$ as the parameter space for our agents. η represents the differentiating parameters for Q-Learning and will be step size α , initial epsilon for epsilon greedy action selection ϵ_0 and epsilon decay factor r . θ represents the parameters for Deep Q-Network, which are number of layers L , nodes per layer N and activation function a . Note that agents only use one of these methods: If an agent uses Q-Learning (Deep Q-Network) then $\theta = \emptyset$ ($\eta = \emptyset$). We also have $\mathcal{P} = \{\pi : \mathcal{S} \rightarrow \mathcal{A}\}$ and

$\mathcal{Q} = \{Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}\}$ as the space of policies and Q-value mappings. For a given agent i , we have a policy $\pi^i \in \mathcal{P}$, Q-value mapping $Q^i \in \mathcal{Q}$, and a learning function $L^i : \mathcal{P} \times \mathcal{Q} \times \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{P} \times \mathcal{Q}$ which transforms our policy and Q-value mapping as we learn. We let $h^i \in \mathcal{H}$ represent the parameters of the learning function.

Evaluation

To evaluate the performance of the model qualitatively, we will use plots of total reward per episode of training (each episode will be trading from 0 to T). On all plots, we will have three series: 1) the upper limit, which plots the best total reward of all agents for each episode, 2) the total rewards for a specific agent per episode, and 3) the lower limit, which plots the worst total reward for that episode. The plots will allow us to examine how the convergence of each agent compares to the worst and best case scenarios. We will have testing episodes, for which we simply use the policies of the agents and no learning occurs, and accordingly we make similar plots of total reward per episode of testing. For quantitative performance, we will use two metrics for each agent: $M_{train}^i = \frac{1}{|E_{train}|} \sum_{e \in E_{train}} \frac{TR_{i,e}}{\sum_{j=1}^n TR_{j,e}}$ and $M_{test}^i = \frac{1}{|E_{test}|} \sum_{e \in E_{test}} \frac{TR_{i,e}}{\sum_{j=1}^n TR_{j,e}}$. These metrics give the average proportion of all total rewards gained by all agents for which agent i themselves gained over the training or testing episodes. Simply put, higher values means means better performance.

References

- [1] [Tidwell 2025](#)
- [2] [Beck 2024](#)
- [3] [Zhang 2023](#)
- [4] [Said 2022](#)